

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Колледж программирования и кибербезопасности

УЧЕБНАЯ ДИСЦИПЛИНА/
МЕЖДИСЦИПЛИНАРНЫЙ КУРС
МДК. 02.01 Технология разработки программного обеспечения

ПЦК ***Программирование***

ВИД УЧЕБНОГО
МАТЕРИАЛА ***Практическое занятие 11-12***

ПРЕПОДАВАТЕЛЬ ***Понеделко Елена Викторовна***

Практическая работа 11-12

Создание диаграммы взаимодействия (диаграммы *последовательности* или последовательности действий и диаграммы ***кооперации)***

Цель: *Изучение нотаций и функциональных возможностей среды для создания диаграмм взаимодействия (*диаграммы *последовательности* или последовательности действий и диаграммы *кооперации*.)

Изучение программного инструмента объектно-ориентированного моделирования StarUML ™, использование функциональных возможностей для создания диаграмм взаимодействия.

Задачи:

- *Изучение программного инструмента объектно-ориентированного моделирования StarUML ™, использование функциональных возможностей для создания диаграмм взаимодействия.*
- *Создание заданных диаграмм взаимодействия* ***(создание диаграммы последовательности и диаграммы кооперации)***

Ход работы

Выполните визуальное моделирование систем с использованием UML на примере проектирования системы заказов интернет-магазина «Style».

Каждый товар в каталоге описывается артикулом, размерным рядом, ценой и фото с кратким описанием.

Покупатель может загрузить каталог товаров. Каталог не содержит разделы, имеет блочную структуру, состоит из набора товаров с фото, ценой и размерами. Покупатель складывает понравившиеся товары в корзину, при этом выбирая размер и количество необходимого товара данного артикула.

Корзину можно изменить: просмотреть, удалить товар, изменить количество позиций одного артикула, вернуться в каталог.

Когда покупатель делает заказ, он вводит свои личные данные, телефон и оплачивает его по банковской карте (если заказ не оплачен, то он и не сделан).

После того как сделан заказ, его можно забрать со склада через 1 рабочий день. Данные о заказе поступают сотруднику магазина, назовем его сотрудником отдела продаж, он проверяет наличие товаров и передает его кладовщику на комплектацию. Кладовщик, собрав заказ, делает отметку о готовности.

Заказ выдается со склада кладовщиком. Кладовщик выдает заказ и отмечает в системе, что заказ выдан.

Магазин не занимается доставкой заказов, не делает скидок. Для того чтобы ограничить масштаб задачи, мы не рассматриваем систему снабжения магазина новыми товарами. Этим занимается другая система, назовем ее Склад. Информация о проданных товарах (т.е. сделанных заказах) поступает также в систему Склад.

Поскольку на протяжении от создания до выдачи заказа, он проходит разные стадии, то будет разумно ввести понятие статуса заказа. Сотрудники магазина могут статус заказа изменять, а покупатель может проследить за сборкой заказа. В таком случае наша система предоставляет еще одну функцию: узнать статус заказа.

Задание 1. **Изучение нотаций и функциональных возможностей среды для создания диаграмм взаимодействия:**

Диаграммы взаимодействия

Диаграммы взаимодействия отображают один из процессов обработки информации в варианте использования: какие объекты нужны потоку, какими сообщениями обмениваются объекты, какие действующие лица инициируют поток и в какой

последовательности отправляются сообщения. Для одного потока событий может быть построено несколько диаграмм взаимодействия.

Основной элемент диаграмм взаимодействия – это объект.

Объектом описывают нечто содержащее в себе данные и поведение. Это термин, описывающий реальные, конкретные предметы или абстрактные сущности.

Объекты изображаются в виде прямоугольников, имена объектов

подчеркиваются. Внутри прямоугольника, обозначающего объект, записывается с большой буквы имя объекта. Имя объекта подчеркивается. Если оно содержит несколько слов, то все они должны начинаться с большой буквы.

Пример. Поскольку наша система предназначена для заказа товаров, то необходимо построить диаграмму ***взаимодействия*** с участием такого объекта, как Товар (например, диаграмму, описывающую добавление товара в корзину). Мы можем использовать на диаграмме взаимодействия какой-то конкретный объект-товар, например, Галстук-Бабочка. А можем назвать этот объект безлично – Товар (см. рис. 1).

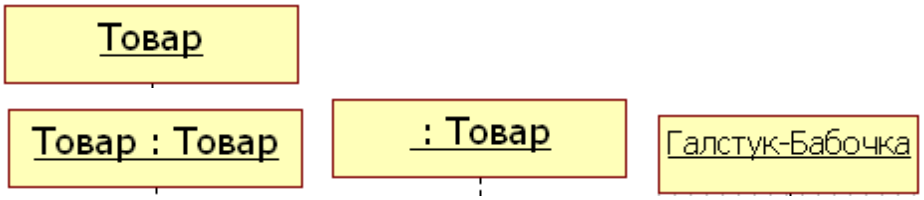


Рисунок 1. Примеры именования объектов

Объекты, помещаемые на диаграммы взаимодействия будут объектами системы и будут относиться к программному обеспечению. При проектировании этих диаграмм можно представлять себе объекты, как экранные формы или части приложений, отвечающие за выполнение определенных действий, или объектом может быть запись в таблице базы данных.

Если *перед тем, как строить диаграммы взаимодействия уже построили диаграммы* классов, то тогда поиск объектов упрощается. Объекты соответствуют своим классам или их операциям, и можно создавать и располагать их на диаграмме последовательности действий или кооперативной диаграмме.

Если мы хотим изучить взаимодействие объектов до того, как переходить к поиску классов, то поиск объектов можно начать с изучения имен существительных в потоке событий. Многие из них станут хорошими кандидатами в объекты. Мы также можем выделить объекты-сущности, граничные объекты и управляющие объекты на основе выбранных классов.

Существует **два типа** диаграмм **взаимодействия** –

- ***диаграммы последовательности (или последовательности действий);***
- ***диаграммы кооперации***.

Первые отображают ***обмен сообщениями между объектами во времени***, а вторые ***отображают структуру взаимодействия***. На обеих диаграммах отображается одна и та же информация, ***но разными способами.***

Диаграмма последовательностей изображает поток управления, а кооперативная диаграмма – поток данных.

Диаграммы последовательности

Как правило, поток событий описывает не одну последовательность действий, а несколько возможных, это отражается наличием главного потока событий и альтернативных потоков. Чаще всего невозможно описать прецедент с помощью только одной последовательности действий. Например, для прецедента **Заказ товаров** возможно оформление заказа без изменения корзины, с изменением состава корзины, или покупатель, просмотрев корзину, захочет вернуться в каталог и что-то в нее добавить, возможно, вернувшись в каталог, покупатель не станет ничего больше добавлять, а снова вернется в корзину и оформит заказ. Каждый такой вариант можно описать своей последовательностью действий, своим сценарием. И, таким образом, *один прецедент*

описывает несколько последовательностей – сценариев, каждый из которых описывает один из вариантов возможного потока событий.

Сценарий (Scenario) – это некоторая последовательность действий, иллюстрирующая поведение системы.

Сценарий – это экземпляр потока событий. Он представляет собой одиночный проход по потоку событий для прецедента. Для графического отображения сценария используются диаграммы последовательностей.

Диаграмма последовательности действий отображает взаимодействие объектов, упорядоченное по времени.

Основные элементы нотации диаграмм последовательности

На диаграммах последовательности изображаются объекты, классы и последовательность сообщений, которыми обмениваются объекты в ходе выполнения сценария (рис. 2).

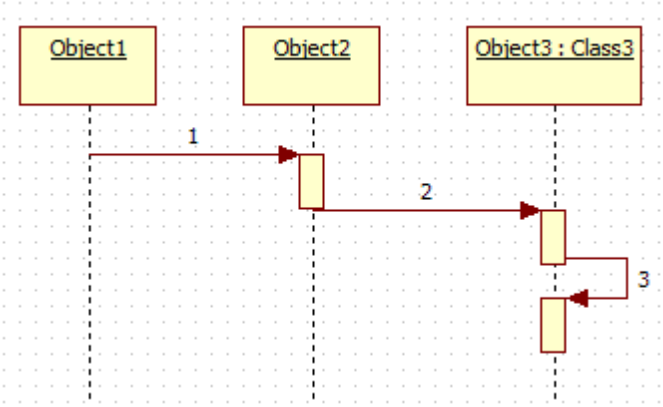


Рисунок 2. Общий вид диаграммы последовательности

На диаграмме последовательностей могут также изображаться экземпляры действующих лиц. *Для того чтобы поместить действующее лицо на диаграмму, нужно найти его в навигаторе модели справа и перетащить на поле диаграммы последовательностей* (рис. 3).

Пример.



Рисунок 3. Экземпляр действующего лица на диаграмме последовательности

Действующие лица, присутствующие на диаграммах взаимодействия, выделяются из потока событий *как сущности, запускающие процессы*. На одной диаграмме их может быть несколько.

Для того чтобы поместить экземпляр уже созданного ранее на диаграмме прецедентов действующего лица на диаграмму взаимодействия, *перетащите его с навигатора модели на рабочее поле диаграммы.*

Каждый объект или действующее лицо на диаграмме последовательностей имеет свою линию жизни, которая обозначается пунктиром.

Линия жизни объекта (object lifeline) – вертикальная пунктирная линия на диаграмме последовательности, которая представляет существование объекта в течение определенного периода времени.

Фокус управления (активность, focus of control) - специальный символ на диаграмме последовательности, указывающий период времени, в течение которого объект выполняет некоторое действие, находясь в активном состоянии.

Фокус управления изображается тонким прямоугольником, расположенным на линии жизни (рис. 4).

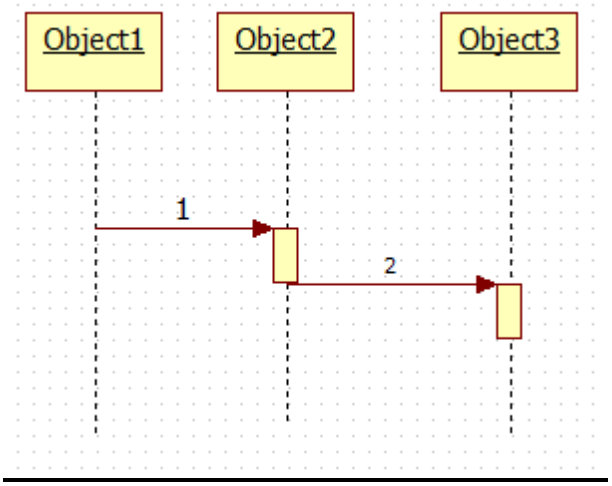


Рисунок 4. Фокус управления

Иногда отображение фокуса активности и нумерации сообщений на диаграмме могут сделать ее трудной для чтения. Чтобы фокус управления и нумерация сообщений не отображались на диаграмме последовательности в StarUML нужно открыть редактор свойств этой диаграммы в инспекторе модели и в разделах ShowSequenceNumber и ShowActivation убрать «галочки» (рис. 5).

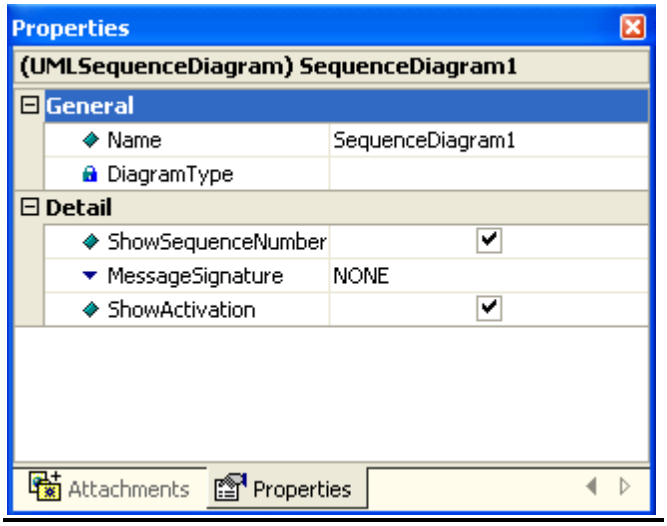


Рисунок 5. Управление отображением фокуса управления и нумерации сообщений

Объекты и действующие лица на диаграммах последовательности обмениваются сообщениями. Сообщения обозначаются стрелками, идущими от отправителя к получателю.

Сообщение (***message***) — спецификация передачи информации от одного элемента модели к другому с ожиданием выполнения определенных действий со стороны принимающего элемента (рис. 6).

Пример.

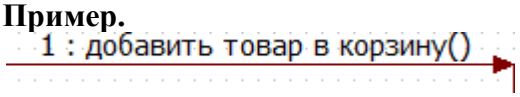


Рисунок 6. Сообщение

Для сообщений на диаграммах последовательностей, как и для других элементов модели, *доступен ряд спецификаций*.

Во-первых, у каждого сообщения должно быть имя, соответствующее его цели.

Во-вторых, сообщения на диаграммах последовательностей можно соотнести с операциями, определенными для классов.

Если от одного объекта к другому направлено сообщение, то это означает, что объект-источник вызывает операцию объекта-приемника. Объект не может вызвать произвольную операцию: она должна быть доступна этому объекту.

В особых случаях сообщение не становится операцией: например, ввод логина и пароля подразумевает их печать в соответствующих полях, и сообщение будет реализовано в виде поля ввода в окне программы.

Процедура создания операций из сообщений будет описана ниже.

В-третьих, мы можем для каждого сообщения установить тип синхронизации. Каждому типу соответствует его обозначение.

Вызов операции (процедуры) (call) вызывает операцию того объекта, к которому направлено. Объект может вызвать свою операцию. Тогда стрелка начинается и заканчивается на линии жизни одного и того же объекта, такое сообщение называется ***рефлексивным***.

Синхронное сообщение обозначается стрелкой с закрашенным наконечником (рис.7).



Рисунок 7. Синхронное сообщение

Асинхронное сообщение (send) посылает объекту сигнал. При этом источник не ждет отклика приемника или подтверждения получения, а продолжает свою работу. Обозначается нежирной стрелкой (рис. 8).



Рисунок 8. Асинхронное сообщение

Ответное сообщение (return) возвращает значение из процедуры тому объекту, к которому направлено. Обозначается пунктирной стрелкой (рис. 9)



Рисунок 9. Ответное сообщение

Создать объект (create) – создает новый объект. Обозначается стрелкой со стереотипом <<create>> (рис. 10).

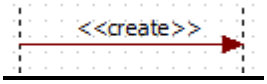


Рисунок 10. Создание объекта

Уничтожить объект (destroy)- удаляет объект. Объект может уничтожить сам себя. Обозначается стрелкой со стереотипом <<destroy>>. При уничтожении объекта на его линии жизни появляется символ разрушения, который обозначается крестом (рис. 11).

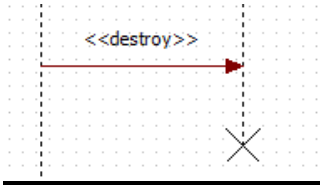


Рисунок 11. Уничтожение объекта

Для *определения типа сообщения* в StarUML нужно выполнить следующие действия: выделите сообщение, щелкнув по соответствующей стрелке один раз левой кнопкой мыши, откройте редактор свойств, выберите на нем раздел ActionKind и в выпадающем списке выберите тот тип синхронизации, который вы хотите установить (рис. 12).

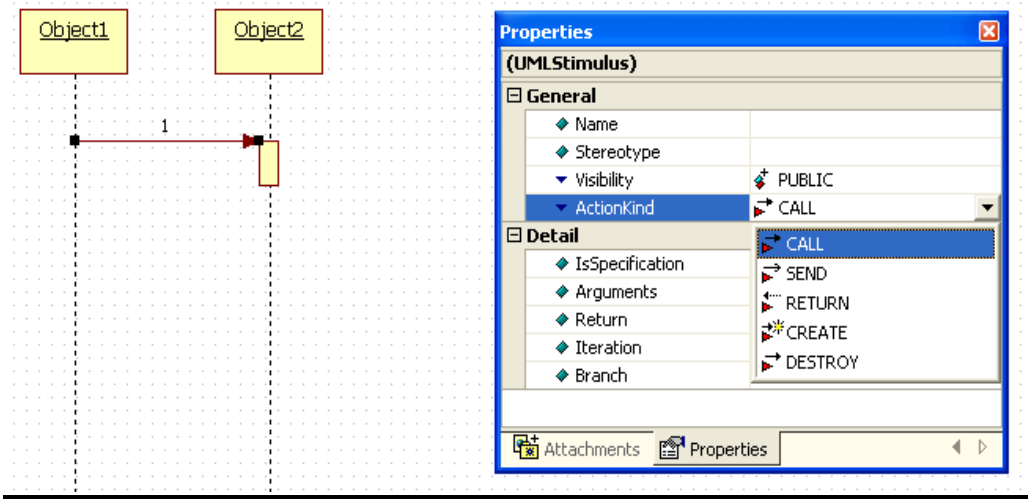


Рисунок 12. Выбор типа сообщения Добавление диаграммы последовательности в модель

Для создания новой диаграммы последовательности нужно выполнить следующие шаги: щелкнуть правой кнопкой мыши по папке представления Logical View в навигаторе модели, в контекстном меню выбрать пункт Add Diagram, в списке выбрать диаграмму последовательности Sequence Diagram (рис. 13).

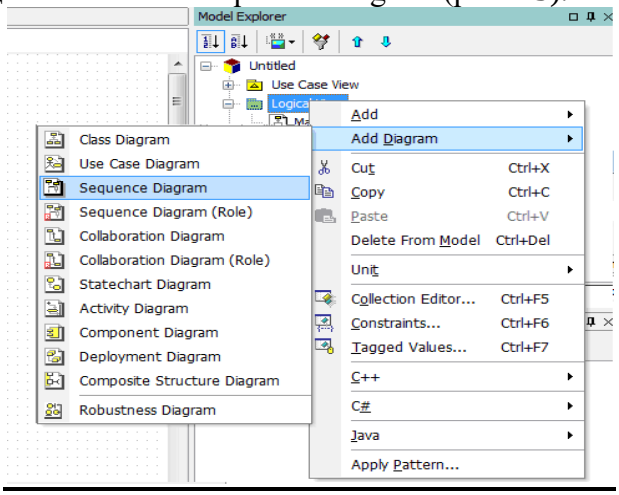


Рисунок 13. Добавление диаграммы последовательности

Диаграмму последовательности можно использовать ***для детализации прецедента.*** Для этого нужно *связать диаграмму с прецедентом*: для создания диаграммы щелкните правой кнопкой мыши по прецеденту, а не по папке Logical View. Однако, если диаграмма последовательности строится для анализа системы, то лучше все-таки помещать ее в Logical View.

Задание 2.Построение диаграммы последовательности

Уже определены классы сценария **Оформление заказа**, теперь с помощью диаграммы последовательности покажем, как взаимодействуют объекты этих классов во времени.

Составьте диаграмму последовательности ***для случая, когда покупатель успешно оформляет заказ*** (рис. 14).

*Покупатель выбирает опцию «**Оформить заказ**» **(place order)**, при этом вызывается некоторый объект **PlaceOrder** (это будет граничный объект, принадлежащий соответствующему граничному классу). Далее открывается форма ввода личных данных покупателя и его кредитной карты **(EnterPersonalInformation)**, на ней покупатель вводит свое имя, адрес, телефон, адрес электронной почты **(enter personal information)** и кредитные данные. Информация принимается и открывается форма подтверждения заказа **(ConfirmOrder)**, покупатель подтверждает, что согласен с реквизитами заказа **(confirm order)**, детали заказа сохраняются для дальнейшего использования **(save the details)**. Фокус управления передается некоторому управляющему объекту **(PlaceOrderManager)**, который обращается к внешней кредитной системе **(Credit System)** для проведения платежа. Если платеж прошел успешно (а именно такой сценарий мы сейчас и рассматриваем), **то PlaceOrderManager** посылает сообщение **(create order)** создать объект **Заказ (Order)**, затем вызывает форму подтверждения заказа **(OrderConfirmation)**. **Объект Заказ (Order)** обращается к объектам **Товар (Item)** для того, чтобы получить информацию о товарах и создает заказ. Процесс завершается.*

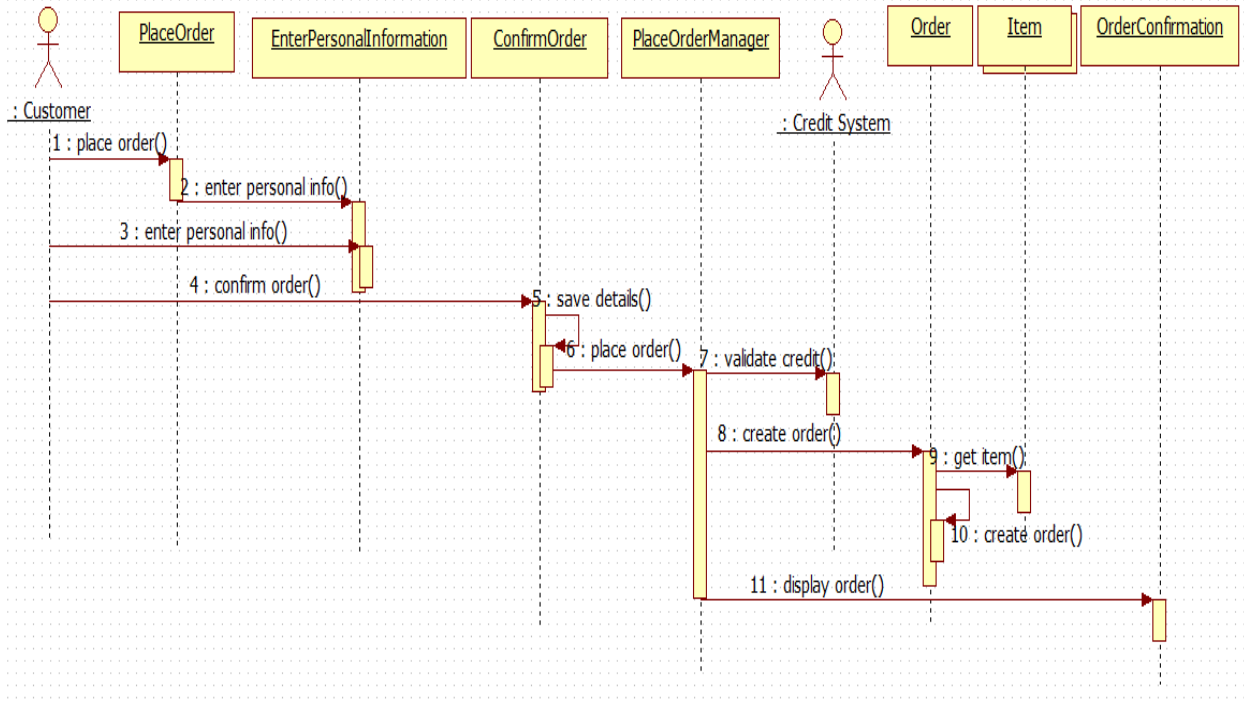


Рисунок 14. Диаграмма последовательности сценария Оформление заказа

Замечание. Обратите внимание, что символ объекта **Товар (Item)** на диаграмме последовательности отличается от символов других объектов. Дело в том, что мы задали множественный экземпляр класса. Действительно, заказ может состоять из нескольких товаров, значит объекту **Заказ (Order)** требуется получить информацию о нескольких объектах **Товар (Item)**. Вместо того, чтобы представлять каждый товар отдельно мы используем нотацию UML для множественного экземпляра класса, представляя одним значком несколько объектов.

Чтобы сделать объект множественным в StarUML, выделите объект, щелкнув по нему мышью один раз, в открывшемся редакторе свойств поставьте флажок в **разделе IsMultiInstance** (рис. 15).

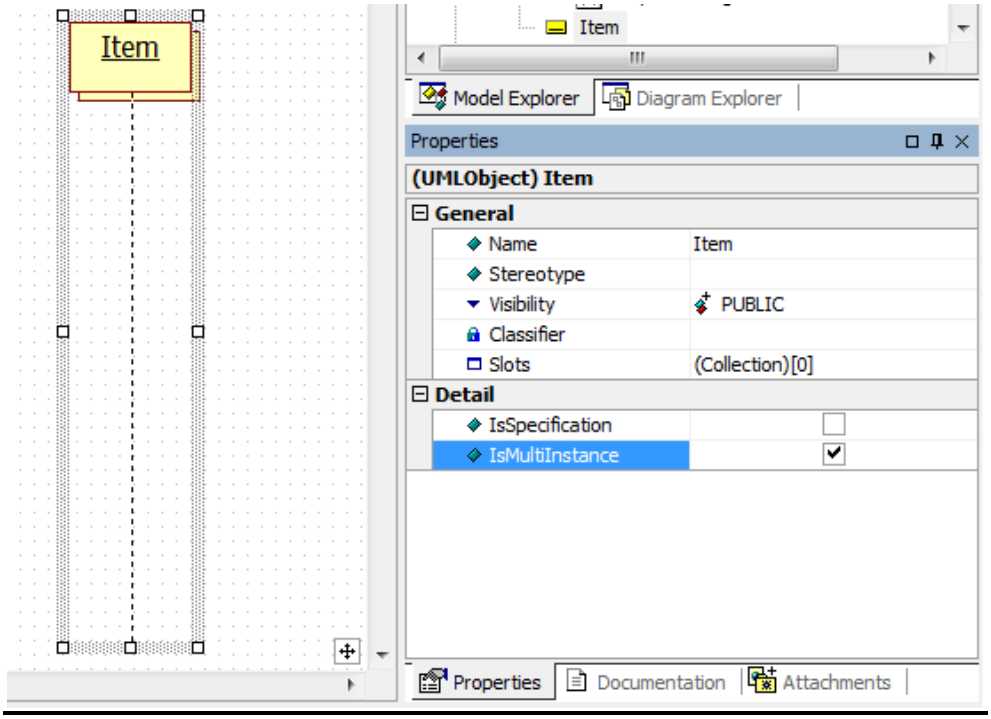


Рисунок 15. Создание множественного объекта

Ветвление потока управления

Обычно для основного потока событий большинства прецедентов строится одна диаграмма последовательности, для альтернативных потоков - дополнительные диаграммы, описывающие все остальные сценарии. Так поступают потому, что на диаграмме последовательности действий сложно отобразить логику **ЕСЛИ-ТО-ИНАЧЕ**. Однако если это необходимо и не загромождает диаграмму, то это можно сделать с помощью условий.

Пример. В процессе оформления покупателем заказа возможны несколько альтернатив. Например, на втором шаге оформления заказа покупатель может подтвердить свой заказ, а может и не согласиться с его реквизитами (см. пример выше). На диаграмме последовательности это можно изобразить так, как это показано на рисунке16.

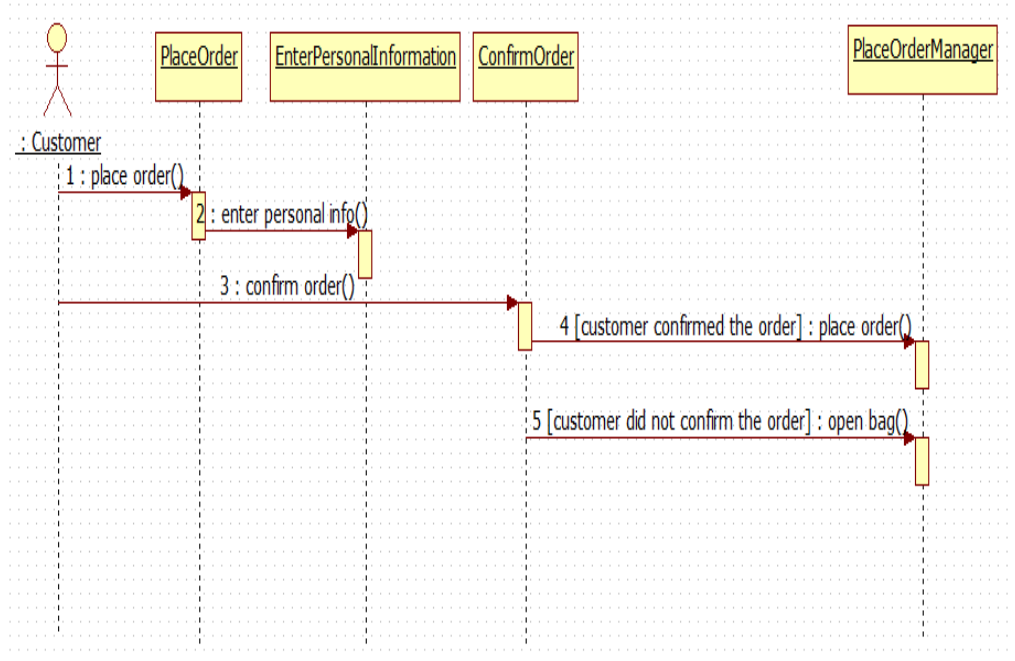


Рисунок 16. Ветвление потока управления

Если покупатель подтверждает свой заказ на втором шаге (**customer confirmed the order)**, то процесс переходит оплате заказа. Если покупатель не подтверждает заказ (**customer did not confirm the order**), то открывается корзина покупателя. Условие, как это принято в нотации UML, записывается в квадратных скобках . Обратите внимание, что мы упростили предыдущую диаграмму описания оформления заказа, иначе добавление ветвей процесса сделало бы ее громоздкой и трудно понимаемой. На практике лучше изображать диаграмму последовательности *отдельно для каждого сценария потока* событий.

Взаимосвязь диаграмм классов и последовательности

Процесс построения модели системы является итеративным. Особенно хорошо это можно видеть при создании диаграмм классов и последовательности.

Какую диаграмму создавать первой: классов или последовательности?

Одни разработчики начинают с диаграмм классов, другие – наоборот, с последовательности. И в том и в другом случае, скорее всего, обе эти диаграммы, построенные для одного сценария, будут в дальнейшем подвергаться изменению. После построения диаграмм последовательности на диаграммах классов могут появиться новые классы, а на диаграммах последовательности – новые объекты, которых раньше там не было, но они придут туда из диаграмм классов. Возможно, что некоторые объекты и классы будут, напротив, удалены.

Внимание! Пример.

В соответствии с нашей диаграммой последовательности на диаграмме классов сценария Оформить заказ произойдут некоторые изменения.

На диаграмме последовательностей покупатель участвует в данном сценарии как действующее лицо-инициатор, запускающий выполнение сценария, но не как внутренний объект системы. Поэтому класс **Customer (Покупатель)** с данной диаграммы классов удалим: такой класс в нашей модели будет (удалили его только с диаграммы), но классом сценария **Оформление заказа** он не является.

Диаграмма классов прецедента **Оформление заказа** изменится и будет выглядеть так (рис. 17).

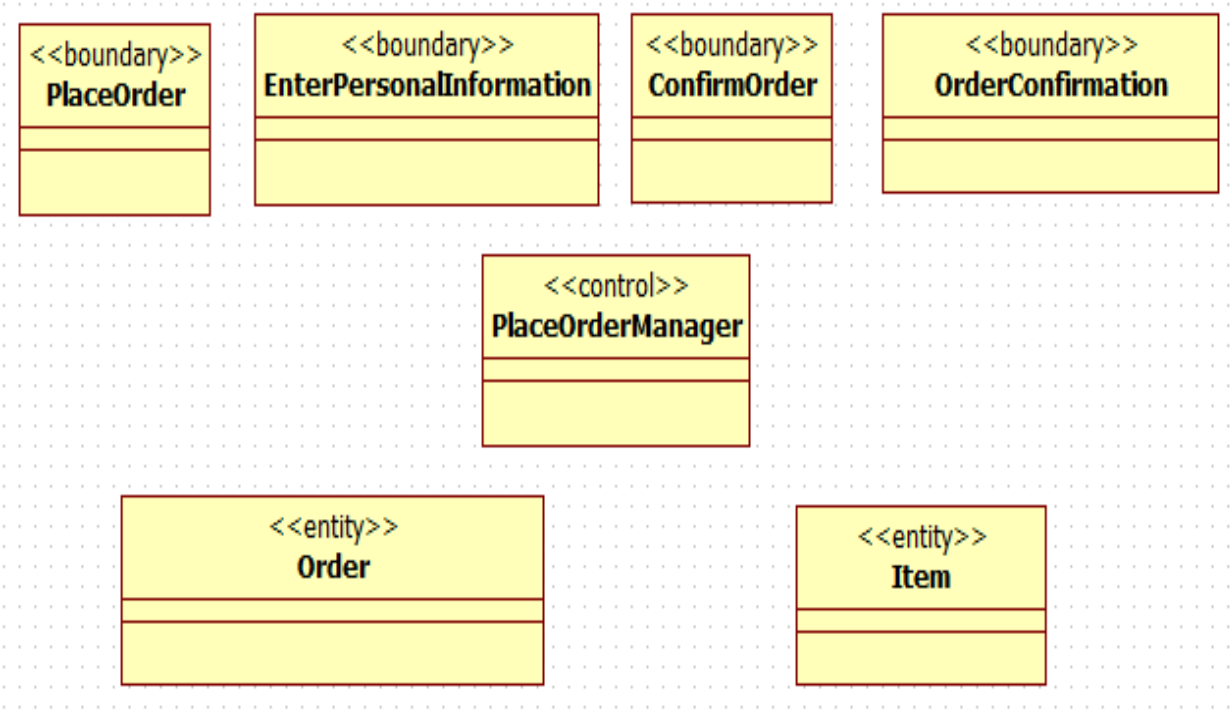


Рисунок 17. Измененная диаграмма классов сценария Оформление заказа

Замечание. Для создания диаграммы последовательностей, *можно было каждый объект этой диаграммы не создавать заново,* а воспользоваться методом перетаскивания. Если перетащить класс с навигатора модели на диаграмму последовательности, то будет создан анонимный объект этого класса (рис. 18).

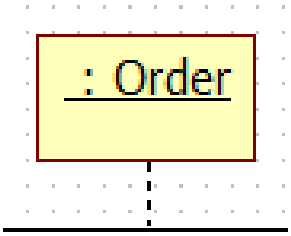


Рисунок 18. Анонимный объект класса Order

Можно изменить имя объекта, присвоив ему имя (рис. 19).

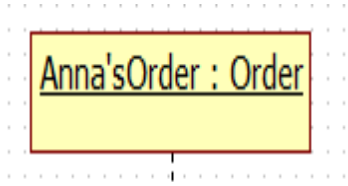


Рисунок 19. Именованный объект класса Order

Задание 3 Постройте диаграмму кооперации

Кооперативные диаграммы

Диаграмма кооперации – это альтернативный способ изображения сценария варианта использования. Этот тип диаграмм заостряет внимание на связях между объектами, отображая обмен данными в системе. А диаграммы последовательности отображают взаимодействие объектов во времени, поэтому ее следует читать сверху вниз и слева направо.

Диаграммы кооперации содержат все те же элементы, что и диаграммы последовательности: объекты, действующие лица, связи между ними и сообщения, которыми они обмениваются, но они уже не упорядочены во времени.

Добавление диаграммы кооперации в модель

Для того чтобы добавить диаграмму кооперации в представление **Logical View**, щелкните правой кнопкой мыши по папке содержащей диаграмму последовательности (если вы ее не переименовывали, то она носит имя **CollaborationInstanceSet1**), в контекстном меню выберите пункт **Add Diagram**, в списке выберите диаграмму кооперации **Collaboration diagram** (рис. 20).

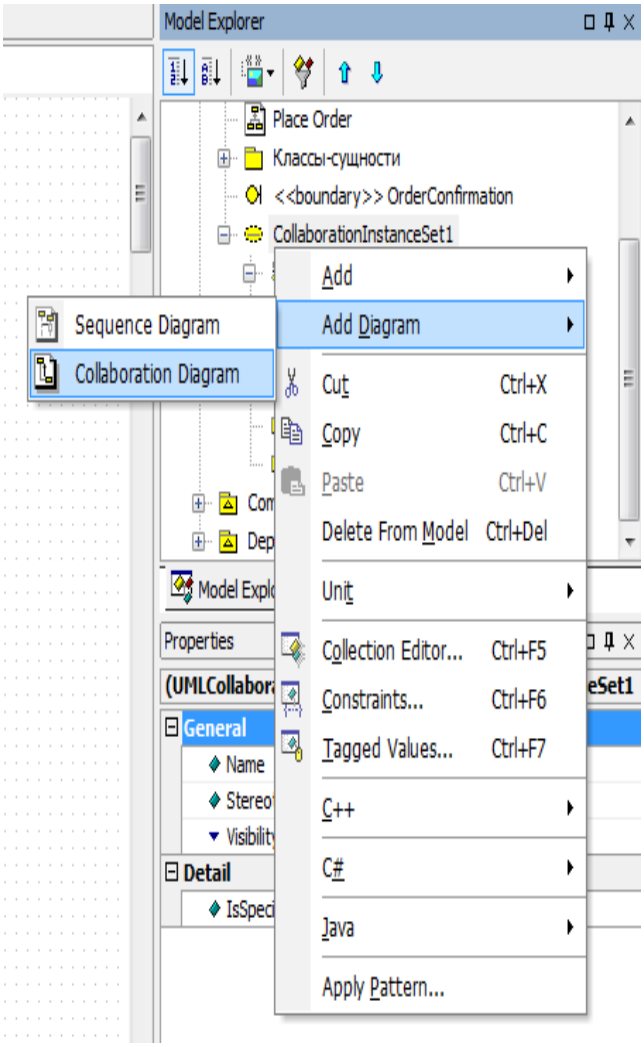


Рисунок 20. Добавление кооперативной диаграммы

Для сценария **Оформление заказа,** для которого уже составили диаграмму последовательности, *на диаграмму кооперации поместите все те же объекты, перетащив их с навигатора модели* (рис. 21).

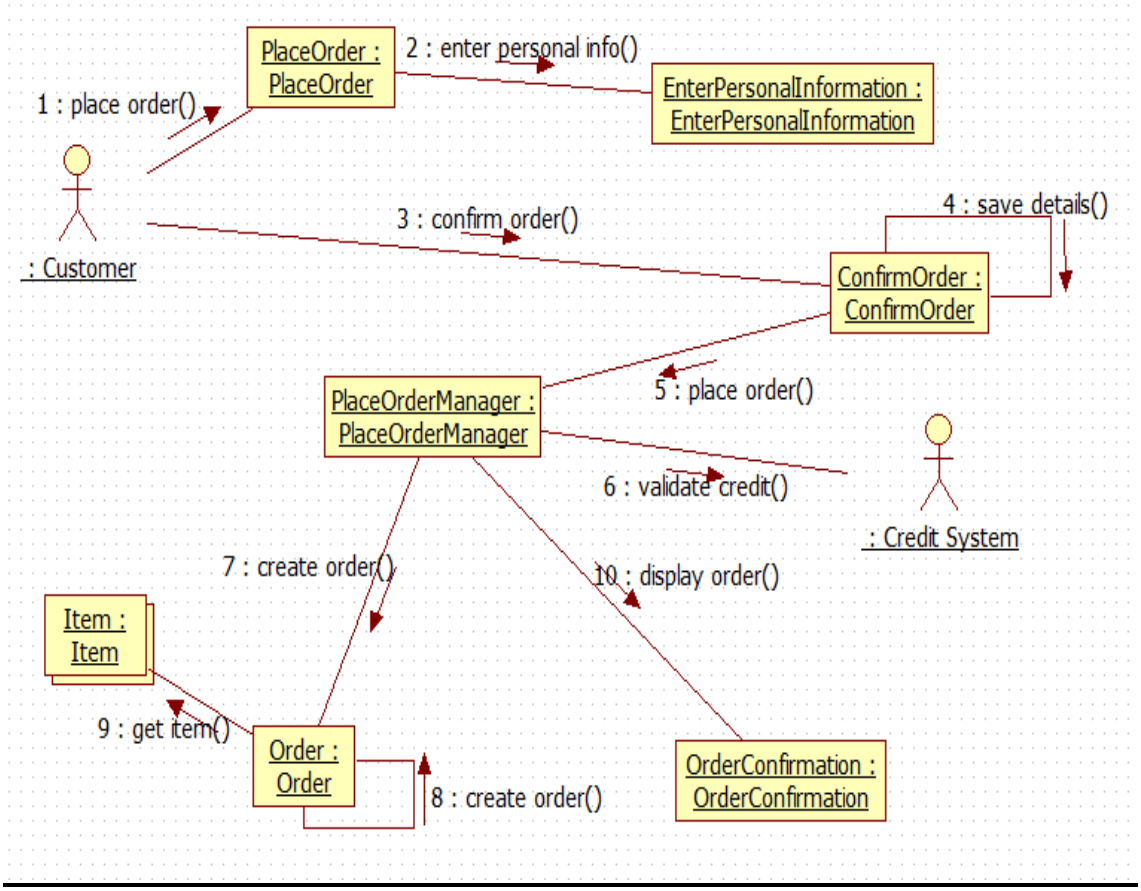


Рисунок 21. Кооперативная диаграмма сценария Оформление заказа

3. <u>Оформить отчет по плану:</u>

1. *Наименование работы*
2. *Цель*
3. *Задачи*
4. *Описание хода работы*
5. *Результат деятельности (скриншот* **Диаграммы последовательности и** ***Диаграммы кооперации)***
6. ***Ответьте на вопросы***

6.1. Назовите основной элемент диаграмм взаимодействия

6.2. Перечислите типы диаграмм взаимодействия и их назначение

6.3. Сценарий (Scenario) – это...

6.4. Действующие лица используются для...

6.5. Что такое Линия жизни объекта (object lifeline)?

6.6. Что такое Фокус управления (активность, focus of control)?

6.7. Виды сообщений (message) и их нотации